

Laboratorio 02: Búsqueda adversaria en espacio de soluciones

18 de agosto de 2026

1. INSTRUCCIONES

1.1. OBJETIVOS

Desarrollar competencias básicas para:

1. Modelar problemas para ser resueltos mediante técnicas de búsqueda adversaria en espacio de soluciones.
2. Implementar algoritmos de búsqueda adversaria: *Min-Max* y *Alpha-Beta*.
3. Resolver problemas de búsqueda adversaria en espacio de soluciones.

1.2. ENTREGABLE

Reglas para el envío de los entregables:

- **Forma de envío:**

Este laboratorio se debe enviar únicamente a través de la plataforma [Moodle](#) en la actividad definida.

- **Archivo de entrega:**

Incluyan en un archivo `.zip` los archivos correspondientes al laboratorio, revise [How to cite in Jupyter Notebooks](#) para citar cualquier recurso utilizado y ver la estructura del proyecto.

- **Nomenclatura para nombrar el archivo de entrega:**

El archivo deberá ser renombrado en mayúsculas con el siguiente formato:
`L02-{color_grupo}-IAIA-{semestre_academico}.zip`

Ejemplos:

`L02-AZUL-IAIA-2026-2.zip`

`L02-ROJO-IAIA-2026-2.zip`

Revise cuando lo requiera [1]

2. I. DESARROLLANDO: BÚSQUEDA CON ADVERSARIO

2.1. A. MODELAR BÚSQUEDA ADVERSARIA

2.1.1. Modelen la estructura de datos para búsqueda adversaria:

Un Game que contenga:

- El *estado* inicial (`initial -> TState`)
- El jugador que tiene el turno en un estado (`player(TState) -> TPlayer`)
- El jugador adversario de un jugador dado (`opponent(TPlayer) -> TPlayer`)
- Si un estado ha finalizado el juego (`is_terminal(TState) -> bool`)
- La *función de transición* de un estado al aplicar las acciones de juego (`result_actions(TState) -> Set[TState]`)
- La *función de utilidad* de un jugador para un estado (`utility(TState, TPlayer) -> float`)

```
[ ]: # =====  
# Modele un juego de un espacio de búsqueda adversaria  
# =====
```

2.2. B. IMPLEMENTAR

2.2.1. Implemente el algoritmo de búsqueda adversaria Min-Max:

1. Implemente la función para el jugador **MAX** (`max_value(Game, TState) -> float`) que devuelve el valor máximo que el jugador **MAX** puede asegurar desde un estado dado.
2. Implemente la función para el jugador **MIN** (`min_value(Game, TState) -> float`) que devuelve el valor mínimo que el jugador **MIN** puede asegurar desde un estado dado.
3. Implemente el algoritmo Min-Max (`minimax_search(Game, TState) -> float`).

```
[ ]: # =====  
# 1. Implemente la función para obtener el valor de un estado para MAX  
# =====
```

```
[ ]: # =====  
# 2. Implemente la función para obtener el valor de un estado para MIN  
# =====
```

```
[ ]: # =====  
# 3. Implemente el algoritmo de búsqueda del Min-Max  
# =====
```

2.2.2. Pruebe la implementación del algoritmo con el siguiente espacio de ejemplo:



1. Implemente la función donde se juega por turnos e inicia un TPlayer dado `play_game(Game, TPlayer) -> Sequence[TState]`
2. Modele la representación del árbol de juego de ejemplo.
3. Use el algoritmo de búsqueda adversaria *Min-Max* para encontrar la utilidad de **MAX** en el estado inicial A del árbol de juego de ejemplo.
4. Presente la secuencia de acciones que tomará J1 para maximizar su utilidad.

Identifique los estados con letras del abecedario A, B, C,

Los jugadores son J1 y J2, **MAX** es el J1

```
[ ]: # =====
# 1. Implemente la función que juega en base a un jugador dado (MAX o MIN)
# =====
def play_game(game, player):
    state = game.initial
    steps = [state]

    while not game.is_terminal(state):
        pass # IMPLEMENTE

    return steps
```

```
[ ]: # =====
# 2. Represente el espacio de búsqueda de ejemplo
# use como referencia el modelado del punto A
# =====
```

```
[ ]: # =====
# 3. Use la implementación del algoritmo de búsqueda min-max
# aplicándola al estado inicial del espacio de búsqueda ejemplo
# =====
```

```
[ ]: # =====
# 4. Use la función `play_game` para jugar como MAX
```

```
# mostrando la secuencia de acciones y la utilidad final
# =====
```

2.2.3. Implemente el algoritmo de búsqueda adversaria Alpha-Beta:

1. Haga modificaciones a la función del jugador MAX e incluya α y β como parámetros adicionales (`max_value(Game, TState, alpha, beta) -> float`)
2. Haga modificaciones a la función del jugador MIN e incluya α y β como parámetros adicionales (`min_value(Game, TState, alpha, beta) -> float`)
3. Implemente el algoritmo Alpha-Beta (`alpha_beta_search(Game, TState) -> float`)
4. Usando la representación del problema de ejemplo previa, ahora pruebe el algoritmo *alpha-beta* en el espacio de ejemplo.

```
[ ]: # =====
# 1. Haga las modificaciones a la función del jugador MAX
# =====
```

```
[ ]: # =====
# 2. Haga las modificaciones a la función del jugador MIN
# =====
```

```
[ ]: # =====
# 3. Implemente el algoritmo de búsqueda alpha-beta
# =====
```

3. II. SOLUCIONAR PROBLEMAS

3.1. A. SOLUCIONANDO

Escoja uno de los siguientes problemas para solucionar: Tic-Tac-Toe, 4 in a row

3.2. A.1 TIC TAC TOE



El objetivo es obtener 3 en fila, horizontal, vertical o diagonal, antes que el adversario. Si todos los espacios se llenan sin un ganador, es empate.

3.3. A.2 4 IN A ROW



El objetivo es obtener 4 en fila antes que el adversario. Cada pieza se coloca en una columna y cae hasta el espacio más bajo disponible. Cuando todas las columnas están llenas sin un ganador, es empate.

1. Presente el diseño del problema y del modelo
2. Implemente los componentes necesarios para solucionar el problema
3. Use el algoritmo de búsqueda correspondiente para solucionar el problema
4. Presente la secuencia de acciones que soluciona el problema usando el algoritmo implementado

```
[ ]: # =====  
# 1. Represente el diseño del problema seleccionado  
# =====
```

```
[ ]: # =====  
# 2. Agregue lo necesario a su representación del problema  
# =====
```

```
[ ]: # =====  
# 3. Use el algoritmo alpha-beta aplicándola al problema de seleccionado  
# =====
```

```
[ ]: # =====  
# 4. Use la función para jugar como MAX  
# mostrando la secuencia de acciones y la utilidad final  
# =====
```

4. III. RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas/Hombre)
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
3. ¿Cuál consideran fue el mayor logro? ¿Por qué?
4. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Referencias

- [1] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach, Global Edition*. Pearson Education, 2021.